

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №11

«Последовательные контейнеры библиотеки STL.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

Задача 1

1. Контейнер - список
2. Тип элементов - double

Задача 2

Тип элементов Pair (см. лабораторную работу №3).

Задача 3

Параметризированный класс – Список (см. лабораторную работу №7)

Задача 4

Адаптер контейнера – очередь с приоритетами.

Задача 5

Параметризированный класс – Список

Адаптер контейнера – очередь с приоритетами.

Задание 3

Найти среднее арифметическое и добавить его в конец контейнера

Задание 4

Найти элементы ключами из заданного диапазона и удалить их из контейнера

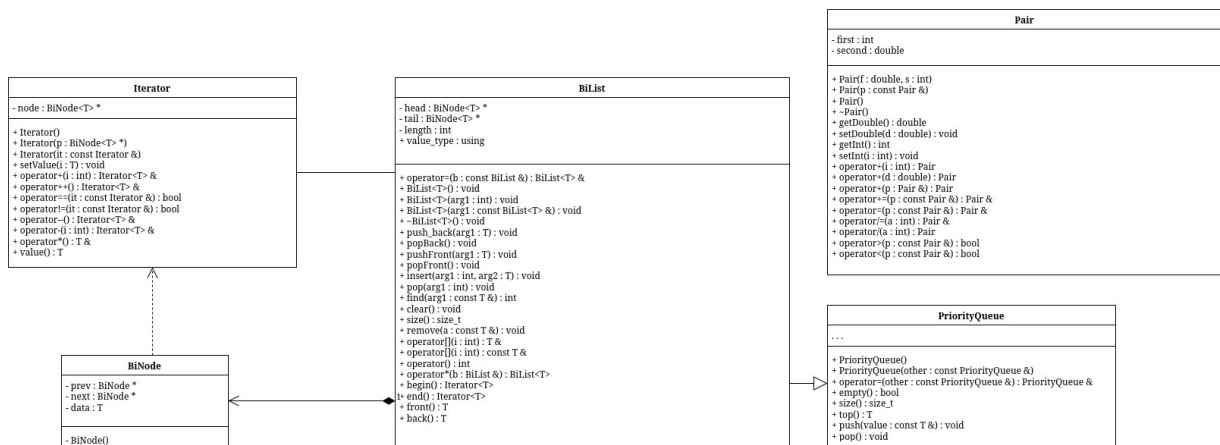
Задание 5

К каждому элементу добавить сумму минимального и максимального элементов контейнер.

АНАЛИЗ.

1. Для каждого задания реализуем template-функции (subtasks). Для всех задач будут использоваться они.
2. Определим необходимые методы и операторы для пользовательских классов, используемых в заданиях.
3. Создадим отдельные функции (tasks) для инициализации всех структур задач и запуска соответствующих заданий.

UML-ДИАГРАММА



KOД bidirectional.hpp

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#include <iosfwd>

template<class T>
class BiList;

template<class T>
std::ostream &operator<<(std::ostream &, BiList<T>);

template<class T>
std::istream &operator>>(std::istream &, BiList<T> &);

template<class T>
class Iterator;

template<class T>
class BiNode {
    friend BiList<T>;
    friend Iterator<T>;
    BiNode *prev;
    BiNode *next;
    T data;

    BiNode() { prev = nullptr, next = nullptr, data = T(); }
};

template<class T>
class Iterator {
    BiNode<T> *node;

public:
    Iterator() { node = nullptr; }
    Iterator(BiNode<T> *p) { node = p; }
    Iterator(const Iterator &it) { node = it.node; }
    bool operator==(const Iterator &it) { return node == it.node; }
    bool operator!=(const Iterator &it) { return node != it.node; };

    Iterator<T> &operator++();

    Iterator<T> &operator--();

    Iterator<T> &operator+(int);
```

```

Iterator<T> &operator-(int);

T &operator*() { return node->data; }
T value() { return node->data; }
void setValue(T i) { node->data = i; }
};

template<class T>
class BiList {
    BiNode<T> *head;
    BiNode<T> *tail;
    int length = 0;

public:
    using value_type = T;

    BiList<T>();

    BiList<T>(int);

    BiList<T>(const BiList<T> &);

    ~BiList<T>() { clear(); }

    void push_back(T);

    void popBack();

    void pushFront(T);

    void popFront();

    void insert(int, T);

    void pop(int);

    int find(const T &);

    void clear();

    size_t size() const;

    void remove(const T &a);

```

```

T &operator[](int);

const T &operator[](int) const;

BiList<T> &operator=(const BiList &);

int operator()() const;

BiList<T> operator*(BiList &);

Iterator<T> begin() const { return Iterator(head); }
Iterator<T> end() const { return Iterator(tail ? tail->next : nullptr); }

T front();

T back();
};

#include "bidirectional.hpp"
#include <iostream>
#include <fstream>

template<class T>
void BiList<T>::push_back(T data) {
    auto node = new BiNode<T>;
    node->data = data;
    if (length == 0) {
        head = node;
        tail = node;
        length++;
        return;
    }

    node->prev = tail;
    tail->next = node;
    tail = node;
    length++;
}

template<class T>
void BiList<T>::popBack() {
    if (length == 0)
        return;

    auto del = tail;

```

```

tail = tail->prev;

if (tail != nullptr)
    tail->next = nullptr;
else
    head = nullptr;

delete del;
length--;
}

```

```

template<class T>
void BiList<T>::pushFront(T data) {
    auto node = new BiNode<T>;
    node->data = data;
    if (length == 0) {
        head = node;
        tail = node;
        length++;
        return;
    }

    node->next = head;
    head->prev = node;
    head = node;
    length++;
}

```

```

template<class T>
void BiList<T>::popFront() {
    if (length == 0)
        return;

    auto del = head;
    head = head->next;

    if (head != nullptr)
        head->prev = nullptr;
    else
        tail = nullptr;

    delete del;
    length--;
}

```

```
}
```

```
template<class T>
void BiList<T>::insert(int idx, T data) {
    if (idx == length) {
        push_back(data);
        return;
    }
    if (idx == 0) {
        pushFront(data);
        return;;
    }

    if (!(0 < idx && idx < length)) {
        return;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    auto newNode = new BiNode<T>;
    newNode->data = data;

    node->prev->next = newNode;
    newNode->prev = node->prev;

    node->prev = newNode;
    newNode->next = node;

    length++;
}
```

```
template<class T>
void BiList<T>::pop(int idx) {
    if (!(0 <= idx && idx < length)) {
        return;
    }
    if (idx == 0) {
        popFront();
        return;
    }
}
```



```

if (idx == length - 1) {
    popBack();
    return;
}

auto node = head;
for (int i = 0; i < idx; i++) {
    node = node->next;
}

node->next->prev = node->prev;
node->prev->next = node->next;

delete node;
length--;
}

```

```

template<class T>
int BiList<T>::find(const T &data) {
    auto node = head;
    int i = 0;
    while (node != nullptr) {
        if (node->data == data) {
            return i;
        }
        i++;
        node = node->next;
    }

    return -1;
}

```

```

template<class T>
void BiList<T>::clear() {
    auto node = head;
    while (node != nullptr) {
        auto tmp = node->next;
        delete node;
        node = tmp;
    }
    head = nullptr;
    tail = nullptr;
    length = 0;
}

```

```

template<class T>
size_t BiList<T>::size() const {
    return length;
}

```

```

template<class T>
void BiList<T>::remove(const T &a) {
    while (find(a) != -1)
        pop(find(a));
}

```

```

template<class T>
T &BiList<T>::operator[](int idx) {
    if (!(0 <= idx && idx < length)) {
        exit(1);
    }

```

```

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

```

```

    return node->data;
}

```

```

template<class T>
const T &BiList<T>::operator[](int idx) const {
    if (!(0 <= idx && idx < length)) {
        exit(1);
    }

```

```

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

```

```

    return node->data;
}

```

```

template<class T>
BiList<T> &BiList<T>::operator=(const BiList<T> &l) {
    if (this == &l)
        return *this;

```

```

        this->clear();
        for (int i = 0; i < l.size(); i++) {
            this->push_back(l[i]);
        }

        return *this;
    }

template<class T>
BiList<T> BiList<T>::operator*(BiList &l) {
    BiList res;
    int range = std::min(l.size(), this->size());
    for (int i = 0; i < range; i++) {
        res.push_back((*this)[i] * l[i]);
    }
    return res;
}

template<class T>
T BiList<T>::front() {
    return head->data;
}

template<class T>
T BiList<T>::back() {
    return tail->data;
}

template<class T>
BiList<T>::BiList() {
    head = nullptr;
    tail = nullptr;
    length = 0;
}

template<class T>
BiList<T>::BiList(int s) {
    head = nullptr;
    tail = nullptr;
    length = 0;
    for (int i = 0; i < s; i++)
        push_back(T());
}

```

```

template<class T>
BiList<T>::BiList(const BiList<T> &l) {
    head = nullptr;
    tail = nullptr;
    length = 0;
    for (int i = 0; i < l.size(); i++) {
        push_back(l[i]);
    }
}

```

```

template<class T>
std::ostream &operator<<(std::ostream &stream, BiList<T> l) {
    for (int i = 0; i < l(); i++)
        stream << l[i] << '\t';
    return stream;
}

```

```

template<class T>
std::istream &operator>>(std::istream &stream, BiList<T> &l) {
    l.clear();
    std::cout << "Size? ";
    int s;
    stream >> s;
    std::cout << "Elements? ";
    for (int i = 0; i < s; i++) {
        T a;
        stream >> a;
        l.push_back(a);
    }
    return stream;
}

```

```

template<class T>
Iterator<T> &Iterator<T>::operator+(int a) {
    for (int i = 0; i < a; i++)
        node = node->next;
    return *this;
}

```

```

template<class T>
Iterator<T> &Iterator<T>::operator-(int a) {
    for (int i = 0; i < a; i++)
        node = node->prev;
    return *this;
}

```

```
}
```

```
template<class T>
```

```
Iterator<T> &Iterator<T>::operator--() {
```

```
    node = node->prev;
```

```
    return *this;
```

```
}
```

```
template<class T>
```

```
Iterator<T> &Iterator<T>::operator++() {
```

```
    node = node->next;
```

```
    return *this;
```

```
}
```

```
#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
```

КОД Pair.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#include <ostream>

class Pair {
    int first;
    double second;

public:
    Pair(double, int);
    Pair(const Pair &);
    Pair();
    ~Pair();
    double getDouble() const;
    void setDouble(double);
    int getInt() const;
    void setInt(int);
    Pair &operator=(Pair const &p);
    Pair operator+(Pair &p);
    Pair operator+(int);
    Pair operator+(double);
    Pair& operator+=(Pair const &p);
    Pair& operator/=(int a);
    Pair operator/(int a) const;
    bool operator>(Pair const &p) const;
    bool operator<(Pair const &p) const;
    bool operator==(const Pair &p) const;
    friend std::ostream &operator<<(std::ostream &, const Pair &);
    friend std::istream &operator>>(std::istream &, Pair &);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
```

КОД Pair.cpp

```
#include "Pair.h"
#include <iostream>

Pair::Pair(double a, int b) {
    first = b;
    second = a;
}

Pair::Pair(const Pair &f) {
    second = f.second;
    first = f.first;
}

Pair::Pair() {
    second = 0;
    first = 0;
}

Pair::~Pair() {
    second = 0;
    first = 0;
}

double Pair::getDouble() const {
    return second;
}

void Pair::setDouble(double a) {
    second = a;
}

int Pair::getInt() const {
    return first;
}

void Pair::setInt(int a) {
    first = a;
}

Pair &Pair::operator=(Pair const &p) {
    second = p.second;
    first = p.first;
    return *this;
}
```

```
}
```

```
bool Pair::operator<(Pair const &p) const {  
    if (this->getInt() < p.getInt())  
        return true;  
    if (this->getInt() == p.getInt())  
        return this->getDouble() < p.getDouble();  
    return false;  
}
```

```
bool Pair::operator>(Pair const &p) const {  
    return p < *this;  
}
```

```
Pair & Pair::operator+=(Pair const &p) {  
    this->first += p.first;  
    this->second += p.second;  
    return *this;  
}
```

```
Pair & Pair::operator/=(int a) {  
    this->first /= a;  
    this->second /= a;  
    return *this;  
}
```

```
Pair Pair::operator/(int a) const {  
    return Pair(  
        this->second / a,  
        this->first / a  
    );  
}
```

```
bool Pair::operator==(const Pair &p) const {  
    return (this->first == p.first && this->second == p.second);  
}
```

```
std::ostream &operator<<(std::ostream &stream, const Pair &p) {  
    std::cout << p.first << ":" << p.second;  
    return stream;  
}
```

```
std::istream &operator>>(std::istream &stream, Pair &p) {  
    std::cout << "Double? ";  
    stream >> p.second;  
}
```



```
std::cout << "Int? ";  
stream >> p.first;  
return stream;  
}
```

```
Pair Pair::operator+(Pair &p) {  
    Pair second(*this);  
    second.second += p.second;  
    second.first += p.first;  
    return second;  
}
```

```
Pair Pair::operator+(int a) {  
    Pair tmp(*this);  
    tmp.first += a;  
    return tmp;  
}
```

```
Pair Pair::operator+(double a) {  
    Pair tmp(*this);  
    tmp.second += a;  
    return tmp;  
}
```

КОД PriorityQueue.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PRIORITYQUEUE_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PRIORITYQUEUE_H
#include "bidirectional.hpp"
```

```
template<class T>
class PriorityQueue : private BiList<T>{
public:
    PriorityQueue() : BiList<T>(){};
    PriorityQueue(const PriorityQueue& other) : BiList<T>(other){};

    PriorityQueue& operator=(const PriorityQueue& other) {
        BiList<T>::operator=(other);
        return *this;
    };

    bool empty() const {
        return !BiList<T>::size();
    };
    size_t size() const {
        return BiList<T>::size();
    };

    T top() {
        T mx = BiList<T>::front();
        for (auto& i : *this)
            if (i > mx) mx = i;
        return mx;
    };

    void push(const T& value) {
        BiList<T>::push_back(value);
    };

    void pop() {
        BiList<T>::remove(top());
    };
};
```

```
#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PRIORITYQUEUE_H
```

КОД main.cpp

```
#include <iostream>
#include <list>
#include <queue>
#include <random>
#include "Pair.h"
#include "bidirectional.hpp"
#include "PriorityQueue.h"

std::random_device rd;
std::mt19937 gen(rd());

template<class T>
void printContainerAfterSubtask(const T& l) {
    std::cout << "\t\t";
    for (auto &i: l) std::cout << i << ' ';
    std::cout << '\n';
}

template<class T>
T subtask1(const T& t) {
    T l = t;
    std::cout << "\t\tsubtask 1\n";
    using Item = typename T::value_type;

    Item avg = Item();
    for (auto &i: l)
        avg += i;
    avg /= l.size();
    l.push_back(avg);

    std::cout << "\t\t" << "avg: " << avg << '\n';

    return l;
}

template<class T>
T subtask2(const T& t) {
    T l = t;

    std::cout << "\t\tsubtask 2\n";
    using Item = typename T::value_type;

    std::uniform_int_distribution<> dist(0, 2);
```

```

BiList<Item> forDelete;
for (auto &i: l)
    if (!dist(gen))
        forDelete.push_back(i);

std::cout << "\t\tFor delete:\n\t\t";
for (auto &i: forDelete) {
    std::cout << i << ' ';
    l.remove(i);
}
std::cout << '\n';

return l;
}

template<class T>
T subtask3(const T& t) {
    T l = t;

    std::cout << "\tsubtask 3\n";
    using Item = typename T::value_type;

    auto mn = l.front(), mx = l.front();
    for (auto &i: l) {
        if (i < mn) mn = i;
        if (i > mx) mx = i;
    }
    Item avg = (mn + mx) / 2;
    for (auto &i: l)
        i += avg;
    std::cout << "\t\t(min + max) / 2 = " << avg << '\n';

    return l;
}

void task_1() {
    std::cout << "task 1\n";

    std::list<double> l;
    std::uniform_int_distribution<> dist(-99, 99);
    for (int i = 0; i < 10; i++)
        l.push_back((double) dist(gen) / 10);

    std::cout << '\t';
    for (double &i: l) std::cout << i << ' ';

```

```

std::cout << '\n';

l = subtask1(l);
printContainerAfterSubtask(l);

l = subtask2(l);
printContainerAfterSubtask(l);

l = subtask3(l);
printContainerAfterSubtask(l);
}

void task_2() {
    std::cout << "task 2\n";

    std::list<Pair> l;
    std::uniform_int_distribution<> dist(0, 9);
    for (int i = 0; i < 10; i++) {
        Pair p((double)dist(gen) / 10, dist(gen));
        l.push_back(p);
    }

    std::cout << '\t';
    for (auto &i: l) std::cout << i << ' ';
    std::cout << '\n';

    l = subtask1(l);
    printContainerAfterSubtask(l);

    l = subtask2(l);
    printContainerAfterSubtask(l);

    l = subtask3(l);
    printContainerAfterSubtask(l);
}

void task_3() {
    std::cout << "task 3\n";

    BiList<Pair> bilist;
    std::uniform_int_distribution<> dist(0, 9);
    for (int i = 0; i < 10; i++) {
        Pair p((double)dist(gen) / 10, dist(gen));
        bilist.push_back(p);
    }
}

```

```

std::cout << '\t';
for (auto &i: bilist) std::cout << i << ' ';
std::cout << '\n';

bilist = subtask1(bilist);
printContainerAfterSubtask(bilist);

bilist = subtask2(bilist);
printContainerAfterSubtask(bilist);

bilist = subtask3(bilist);
printContainerAfterSubtask(bilist);
}

template<class T>
std::priority_queue<T> copyBiListToSTLPriorityQueue(const BiList<T> & bl) {
    std::priority_queue<T> q;
    for (auto& i : bl)
        q.push(i);
    return q;
}

template<class T>
BiList<T> copySTLPriorityQueueToBiList(std::priority_queue<T> & q) {
    BiList<T> bl;
    while (!q.empty()) {
        bl.push_back(q.top());
        q.pop();
    }
    q = copyBiListToSTLPriorityQueue(bl);
    return bl;
}

template<class T>
void printSTLPriorityQueue(std::priority_queue<T> & q) {
    auto v = copySTLPriorityQueueToBiList(q);
    for (auto &i: v) std::cout << i << ' ';
    std::cout << '\n';
}

void task_4() {
    std::cout << "task 4\n";
    std::priority_queue<Pair> q;

```

```

std::uniform_int_distribution<> dist(0, 9);
for (int i = 0; i < 10; i++) {
    Pair p((double)dist(gen) / 10, dist(gen));
    q.push(p);
}

std::cout << '\t';
printSTLPriorityQueue(q);

auto v = copySTLPriorityQueueToBiList(q);
v = subtask1(v);
q = copyBiListToSTLPriorityQueue(v);
std::cout << "\t\t";
printSTLPriorityQueue(q);

v = copySTLPriorityQueueToBiList(q);
v = subtask2(v);
q = copyBiListToSTLPriorityQueue(v);
std::cout << "\t\t";
printSTLPriorityQueue(q);

v = copySTLPriorityQueueToBiList(q);
v = subtask3(v);
q = copyBiListToSTLPriorityQueue(v);
std::cout << "\t\t";
printSTLPriorityQueue(q);

std::cout << '\n';
}

template<class T>
PriorityQueue<T> copyBiListToMyPriorityQueue(const BiList<T> & bl) {
    PriorityQueue<T> q;
    for (auto& i : bl)
        q.push(i);
    return q;
}

template<class T>
BiList<T> copyMyPriorityQueueToBiList(PriorityQueue<T> & q) {
    BiList<T> bl;
    while (!q.empty()) {
        bl.push_back(q.top());
    }
}

```

```

        q.pop();
    }
    q = copyBiListToMyPriorityQueue(bl);
    return bl;
}

```

```

template<class T>
void printMyPriorityQueue(PriorityQueue<T> & q) {
    auto v = copyMyPriorityQueueToBiList(q);
    for (auto &i: v) std::cout << i << ' ';
    std::cout << '\n';
}

```

```

void task_5() {
    std::cout << "task 5\n";
    PriorityQueue<Pair> q;

    std::uniform_int_distribution<> dist(0, 9);
    for (int i = 0; i < 10; i++) {
        Pair p((double)dist(gen) / 10, dist(gen));
        q.push(p);
    }

    std::cout << '\t';
    printMyPriorityQueue(q);

    auto v = copyMyPriorityQueueToBiList(q);
    v = subtask1(v);
    q = copyBiListToMyPriorityQueue(v);
    std::cout << "\t\t";
    printMyPriorityQueue(q);

    v = copyMyPriorityQueueToBiList(q);
    v = subtask2(v);
    q = copyBiListToMyPriorityQueue(v);
    std::cout << "\t\t";
    printMyPriorityQueue(q);

    v = copyMyPriorityQueueToBiList(q);
    v = subtask3(v);
    q = copyBiListToMyPriorityQueue(v);
    std::cout << "\t\t";
    printMyPriorityQueue(q);
}

```



```
std::cout << '\n';  
}
```

```
int main() {  
    task_10;  
    task_20;  
    task_30;  
    task_40;  
    task_50;  
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

task 1

-1.3 -5.3 7.7 5.5 -4.4 -7 -3.5 -7.8 -4.4 6.9

subtask 1

avg: -1.36

-1.3 -5.3 7.7 5.5 -4.4 -7 -3.5 -7.8 -4.4 6.9 -1.36

subtask 2

For delete:

7.7 -7

-1.3 -5.3 5.5 -4.4 -3.5 -7.8 -4.4 6.9 -1.36

subtask 3

$(\min + \max) / 2 = -0.45$

-1.75 -5.75 5.05 -4.85 -3.95 -8.25 -4.85 6.45 -1.81

task 2

0:0.6 7:0.7 8:0.9 7:0.2 9:0 9:0.5 9:0.3 3:0.3 2:0.8 4:0.5

subtask 1

avg: 5:0.48

0:0.6 7:0.7 8:0.9 7:0.2 9:0 9:0.5 9:0.3 3:0.3 2:0.8 4:0.5 5:0.48

subtask 2

For delete:

0:0.6 8:0.9 7:0.2 9:0 4:0.5

7:0.7 9:0.5 9:0.3 3:0.3 2:0.8 5:0.48

subtask 3

$(\min + \max) / 2 = 5:0.65$

12:1.35 14:1.15 14:0.95 8:0.95 7:1.45 10:1.13

task 3

0:0.8 0:0.5 3:0.4 9:0.1 7:0.5 9:0 7:0.9 6:0.9 7:0.3 0:0.8

subtask 1

avg: 4:0.52

0:0.8 0:0.5 3:0.4 9:0.1 7:0.5 9:0 7:0.9 6:0.9 7:0.3 0:0.8 4:0.52

subtask 2

For delete:

7:0.5 9:0 7:0.3 4:0.52

0:0.8 0:0.5 3:0.4 9:0.1 7:0.9 6:0.9 0:0.8

subtask 3

$(\min + \max) / 2 = 4:0.3$

4:1.1 4:0.8 7:0.7 13:0.4 11:1.2 10:1.2 4:1.1

task 4

9:0.3 8:0.2 6:0.5 6:0.3 3:0.7 3:0.6 2:0.6 2:0.4 1:0.2 0:0.6

subtask 1

avg: 4:0.44

9:0.3 8:0.2 6:0.5 6:0.3 4:0.44 3:0.7 3:0.6 2:0.6 2:0.4 1:0.2 0:0.6

subtask 2

For delete:

2:0.6 2:0.4 1:0.2

9:0.3 8:0.2 6:0.5 6:0.3 4:0.44 3:0.7 3:0.6 0:0.6

subtask 3

$$(\min + \max) / 2 = 4:0.45$$

13:0.75 12:0.65 10:0.95 10:0.75 8:0.89 7:1.15 7:1.05 4:1.05

task 5

9:0.5 8:0.8 7:0 4:0.6 4:0.1 3:0.2 3:0.1 2:0.8 2:0.6 1:0.5

subtask 1

avg: 4:0.42

9:0.5 8:0.8 7:0 4:0.6 4:0.42 4:0.1 3:0.2 3:0.1 2:0.8 2:0.6 1:0.5

subtask 2

For delete:

9:0.5 4:0.42 3:0.2 2:0.8 2:0.6

8:0.8 7:0 4:0.6 4:0.1 3:0.1 1:0.5

subtask 3

$$(\min + \max) / 2 = 4:0.65$$

12:1.45 11:0.65 8:1.25 8:0.75 7:0.75 5:1.15

ВОПРОСЫ

1. Из каких частей состоит библиотека STL?

Контейнеры, итераторы

2. Какие типы контейнеров существуют в STL?

Последовательные, ассоциативные, адаптеры контейнеров

3. Что нужно сделать для использования контейнера STL в своей программе?

Подключить соответствующий заголовочный файл и указать тип элементов через шаблон

4. Что представляет собой итератор?

Итератор — более общее понятие, чем указатель, используемое для доступа к элементам контейнера

5. Какие операции можно выполнять над итераторами?

Разыменование (*), присваивание, сравнение (==, !=), инкремент (++)

6. Каким образом можно организовать цикл для перебора контейнера с использованием итератора?

for (i = first; i != last; i++)

7. Какие типы итераторов существуют?

Входные, выходные, прямые, двунаправленные, итераторы произвольного доступа

8. Перечислить операции и методы общие для всех контейнеров.

==, !=, =, clear, insert, erase, size, max_size, empty, begin, end, reverse begin/end

9. Какие операции являются эффективными для контейнера vector? Почему?

Доступ по [] и at, push_back; потому что элементы в непрерывной памяти

10. Какие операции являются эффективными для контейнера list? Почему?

Вставка и удаление в любой позиции; так как это двусвязный список

11. Какие операции являются эффективными для контейнера deque? Почему?

Вставка и удаление в начале и конце; так как поддерживает работу с обоими концами

12. Перечислить методы, которые поддерживает последовательный контейнер vector.

push_back, pop_back, insert, erase, [], at, swap, clear

13. Перечислить методы, которые поддерживает последовательный контейнер list.

push_back, pop_back, push_front, pop_front, insert, erase, splice, swap, clear

14. Перечислить методы, которые поддерживает последовательный контейнер deque.

push_back, pop_back, push_front, pop_front, insert, erase, [], at, swap, clear

15. Задан контейнер vector. Как удалить из него элементы со 2 по 5?

`v.erase(v.begin() + 1, v.begin() + 5)`

16. Задан контейнер vector. Как удалить из него последний элемент?

`v.erase(v.end() - 1)`

17. Задан контейнер list. Как удалить из него элементы со 2 по 5?

`v.erase(next(v.begin(),1), next(v.begin(),5))`

18. Задан контейнер list. Как удалить из него последний элемент?

`v.erase(--v.end())`

19. Задан контейнер deque. Как удалить из него элементы со 2 по 5?

`d.erase(d.begin() + 1, d.begin() + 5)`

20. Задан контейнер deque. Как удалить из него последний элемент?

`d.erase(d.end() - 1)`

21. Написать функцию для печати последовательного контейнера с использованием итератора.

```
template<class T> void print(char* String, T& C) { T::iterator p = C.begin(); for(; p != C.end(); ++p)
cout << *p << " "; }
```

22. Что представляют собой адаптеры контейнеров?

Специализированные контейнеры, реализованные на основе базовых контейнеров

23. Чем отличаются друг от друга объявления `stack<int> s` и `stack<int, list<int> > s`?

Во втором случае стек реализован на основе списка, а в первом — по умолчанию на deque

24. Перечислить методы, которые поддерживает контейнер stack.

push, pop, top, empty, size

25. Перечислить методы, которые поддерживает контейнер queue.

push, pop, front, back, empty, size

26. Чем отличаются друг от друга контейнеры queue и priority_queue?

priority_queue возвращает максимальный элемент, queue работает по принципу FIFO

27. Задан контейнер stack. Как удалить из него элемент с заданным номером?

Использовать дополнительный стек-буфер: последовательно извлечь элементы из исходного stack во временный stack до нужного элемента, удалить его (не помещать обратно), затем вернуть элементы обратно в исходный стек.

28. Задан контейнер queue. Как удалить из него элемент с заданным номером?

Использовать дополнительную очередь-буфер: последовательно переносить элементы из исходной queue в вспомогательную queue до нужного элемента, пропустить его, затем продолжить перенос остальных элементов обратно

29. Написать функцию для печати контейнера stack с использованием итератора.

```
template <class T>
void printStack(stack<T> s)
{
    vector<T> buf;

    while (!s.empty()) {
        buf.push_back(s.top());
        s.pop();
    }

    for (vector<T>::iterator i = buf.begin(); i != buf.end(); i++)
        cout << *i << " ";

    cout << '\n';

    for (int i = buf.size() - 1; i >= 0; i--)
        s.push(buf[i]);
}
```

30. Написать функцию для печати контейнера queue с использованием итератора.

```
template <class T>
void printQueue(queue<T> q)
{
    vector<T> buf;

    while (!q.empty()) {
        buf.push_back(q.front());
        q.pop();
    }

    for (vector<T>::iterator it = buf.begin(); it != buf.end(); ++it)
        cout << *it << " ";

    cout << '\n';

    for (size_t i = 0; i < buf.size(); ++i)
        q.push(buf[i]);
}
```